Lifelong Learning for an Origami Robot: Supervised Learning of Target Adaptations

E. De Vroey

***Abstract*—Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.**

I. Introduction

II. Background

A. Lifelong Learning

B. Kresling Origami

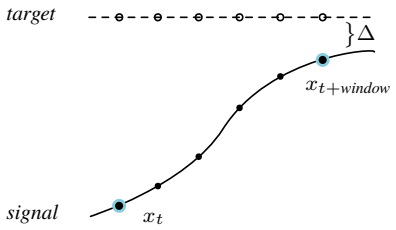
III. Methodology

A. Target Adaptations

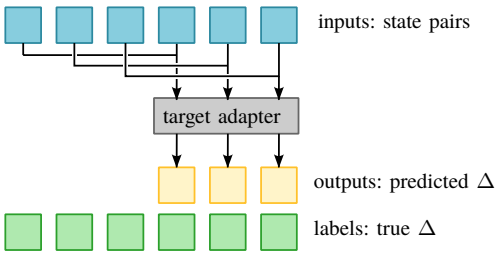
The method proposed in this paper aims to provide a way of improving a given controller's performance as well as fortifying it against evolving system dynamics *without* having access to the inner workings of the controller itself. In other words, as long as the fundamental assumptions of a controller are satisfied—i.e. it requires a state and a target as input and yields a control action—the method can remain completely agnostic to the specific controller being used. In addition, the aim is not to replace the controller entirely but to introduce an additional module to the control scheme that enables lifelong robustness to evolving system dynamics. The advantage of such a method is that it could be applied to any system as a *plug-and-play* module.

The manner in which this will be achieved is by *target adaptation*. An *adapter* module modifies the target used by the controller to improve its performance. Consider a scenario where a controller yields a control action u to reach a certain desired target state x_d. As the system dynamics evolve, suppose this action u is now insufficient to reach the target. In response, the adapter module *lies* to the controller about reaching x_d and instead provides it with $x_d + \Delta$, for which the controller yields a larger action u^* that *does* result in the system reaching state x_d.

The *adapter* module is implemented as a feedforward neural network. To train this model to learn suitable target adaptations, interactions of the controller with the system are recorded. The model is then provided with pairs of system states, spaced by a certain time window x_t and $x_{t+\text{window}}$, that are obtained from these recordings. From these pairs, the model infers the target it 'believes' the controller aims to reach. However, instead of predicting the target directly, the model learns the difference Δ between the target and the latter state in the pair (figs. 1a and 1b).



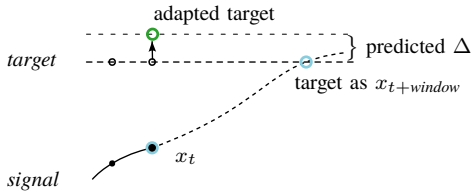
(a) The target adaptation problem: Based on the current x_t and the state some prediction window later $x_{t+window}$, the model learns to predict the target the controller is trying to reach by representing it as a Δ to be added to $x_{t+window}$.



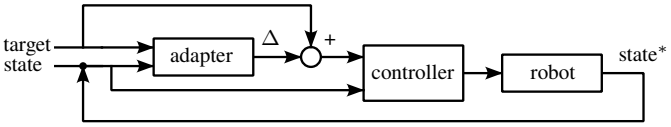
(b) The training process of the target adapter: recorded states spaced by a specified prediction window are paired up and provided as features to the adapter, which learns to predict a Δ, being the difference between the target and state $x_{t+window}$.

Fig. 1: Target adaptation during the training phase

The model is trained in a supervised manner (fig. 1b) and then deployed. During deployment, it is presented with the current state x_t but cannot be presented with $x_{t+window}$ (since this state is not known yet). Instead, the second state in the pair is set to the desired target. To the model, this represents a situation in which the desired target was reached from the current state within the specified time window. The idea is now that the trained model will produce a Δ that, when added to the latter state in the pair (i.e., the desired target), results in an adapted target (fig. 2a) for which the controller will produce a control action. In short, given a current state x_t and a future state $x_{t+window}$, the model learned to produce the desired target. Thus, when presented with x_t and $x_{t+window} = x_d$ (supposedly reached within the time window) it provides the (adapted) target that would *push* the controller toward that result. A full overview is presented in figs. 2a and 2b.



(a) Target adaptation during deployment: By supplying the model with the controller's target as $x_{t+window}$ it yields a Δ that can be added to the current target, which will supposedly result in the controller reaching the target within the specified window.



(b) Schematic overview of the full implementation: The adapter is used to offset the given target before yielding it to the controller, essentially *lying* to the controller in order to improve its performance.

Fig. 2: Target adaptation during deployment.

The reason for predicting the difference Δ between the state $x_{t+window}$ and the desired target, rather than a direct prediction of the target, can also be clarified now: By designing the model's output to be a Δ, the model can be initialized with parameters all close to zero (essentially leaving the desired target unchanged) and then steadily improve online as interactions of the controller and system are recorded.

B. Toy Problem

To explore the proposed method, a toy problem is considered first. The toy problem consists of a simulation of a simple 1D mass-spring-damper system and an imperfectly tuned PID controller (in this case, it is too aggressive). With intervals of 5 seconds, a series of random targets are generated for the controller to reach and maintain. As the simulation progresses, the parameters of the dynamic system are slowly changed to simulate a degradation in the system.

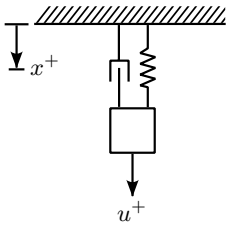


Fig. 3

Algorithm 1: Toy problem target adaptation

Data: system state x_0, target x_d, adaption Δ, control action u
Initialize model: *adapter*;
Initialize buffer: *buffer* $\leftarrow []$;
for *each timestep* t_i **do**
  if *len(buffer) > threshold* **then**
    Remove the oldest element in the buffer;
  end
  if $t_i \mod target\_interval = 0$ **then**
    $x_d \leftarrow random\_target()$;
  end
  if $t_i \mod update\_interval = 0$ **then**
    (*features, labels*) $\leftarrow$ *prep_data(buffer, window)*;
    adapter.fit(features, labels);
  end
  $\Delta \leftarrow adapter.predict([x_0, x_d])$;
  $u \leftarrow controller.compute\_control(x_0, x_d + \Delta)$;
  Simulate the system response x;
  Update system parameters;
  Append $[x_0, u, x, x_d + \Delta]$ to *buffer*;
end

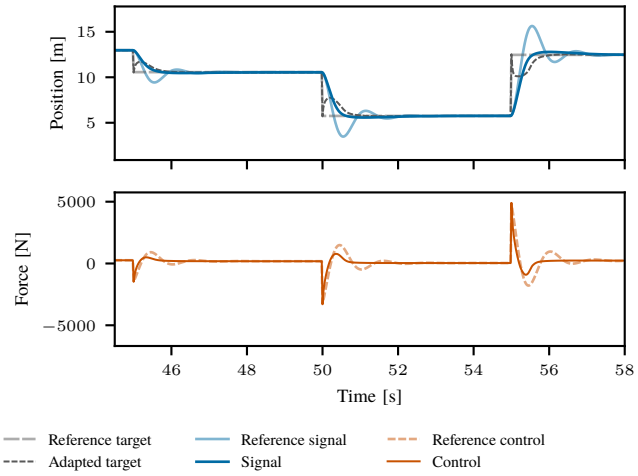


Fig. 4

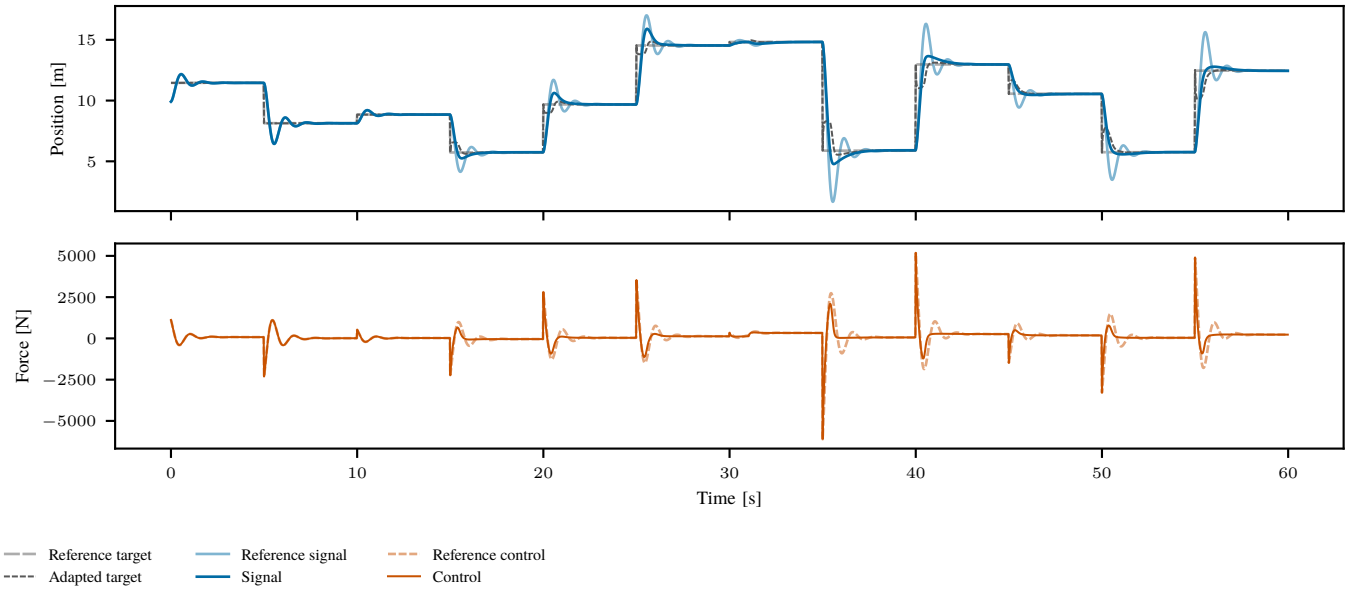


Fig. 5